CO 2 - AT 1

C PROGRAMMING

Name	S Dhana Sri
Reg No	19252620062
Course	CSA0209
Slot	B
Github repository	dhanasri-saravanan/1925260062-S-Dhana-sri-C-Programming

Q1. Precedence, Associativity and the Discount Bug

Problem: Calculate price × quantity, apply a 15% senior-citizen discount, add 5% GST, and subtract Rs. 50 when the amount is above Rs. 1000.

Precedence and sample evaluation

In C, multiplication and division are evaluated before addition and subtraction. Operators of equal precedence associate left to right. In the given expression, the 15% discount term is multiplied by the senior flag; the GST term is added afterward. For price = 100, qty = 2, senior = 1: base = 200, discount = 30, GST term = 10, and the original expression total = Rs. 180. The Rs. 50 reduction does not apply.

Undefined behaviour

The statement count = count++ + 1; modifies count through both post-increment and assignment without the required sequencing. Its behaviour is undefined, so a compiler is not required to produce a predictable value. Use count++; or count = count + 1; as a separate statement.

Corrected C program

This version calculates GST on the discounted subtotal, a common billing interpretation. If the assignment expects GST on the original base as in its expression, change gst = taxable * 0.05; to gst = base * 0.05;.

```
#include <stdio.h>
int main(void)
{
    double price = 100.0;
    int qty = 2, senior = 1;
    double base, discount, taxable, gst, total, net;

    base = price * qty;
    discount = senior ? base * 0.15 : 0.0;
    taxable = base - discount;
    gst = taxable * 0.05;
    total = taxable + gst;
    net = (total > 1000.0) ? total - 50.0 : total;

    printf("Base amount : Rs. %.2f\n", base);
    printf("Discount    : Rs. %.2f\n", discount);
    printf("GST         : Rs. %.2f\n", gst);
    printf("Net payable : Rs. %.2f\n", net);
    return 0
}
```

EXECUTED CODE:

Code Input Output

```c
// #include <stdio.h>
int main(void)
{
    double price = 100.0;
    int qty = 2, senior = 1;
    double base, discount, taxable, gst, total, net;

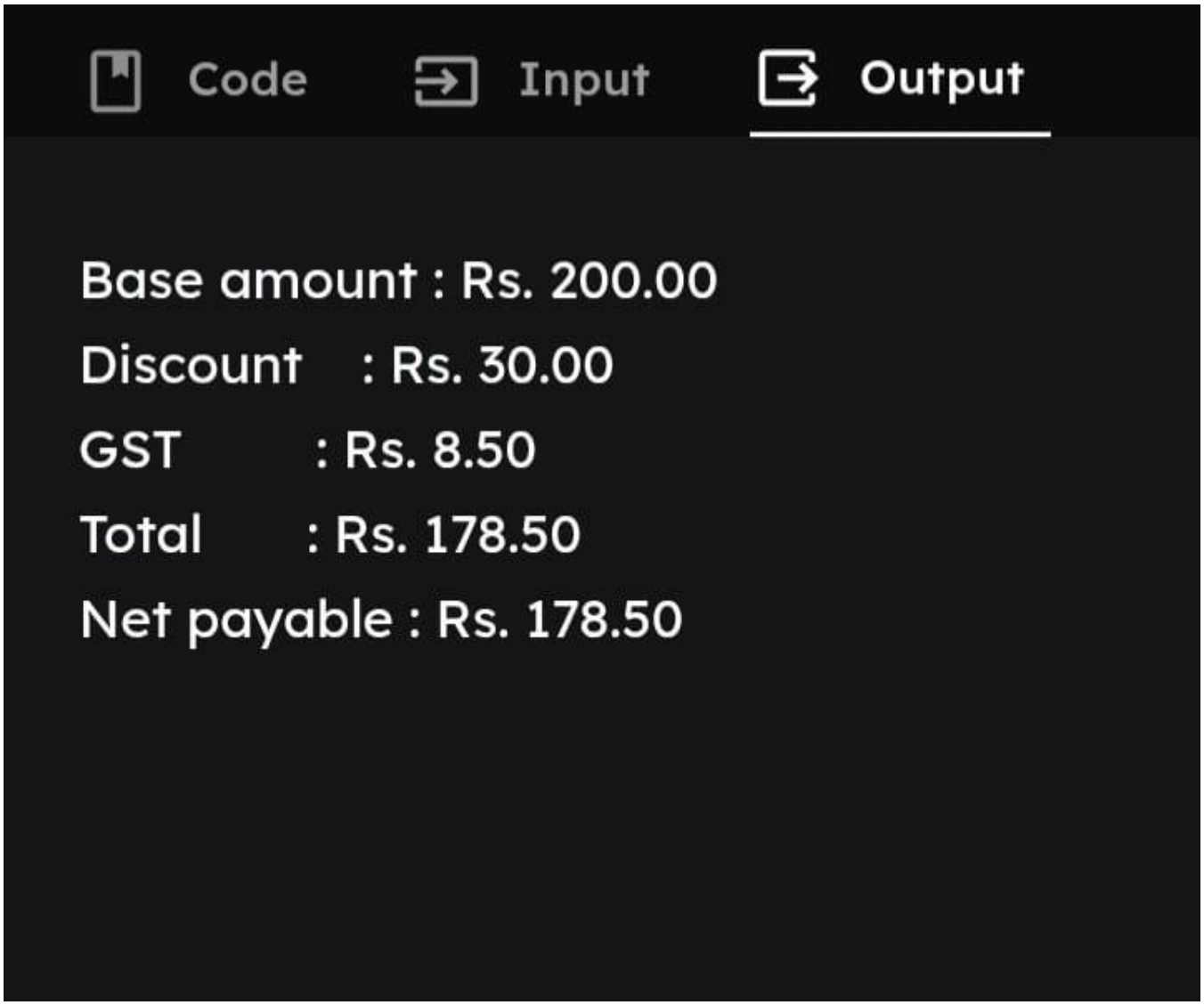
    base = price * qty;
    discount = senior ? base * 0.15 : 0.0;
    taxable = base - discount;
    gst = taxable * 0.05;
    total = taxable + gst;
    net = (total > 1000.0) ? total - 50.0 : total;

    printf("Base amount : Rs. %.2f\n", base);
    printf("Discount    : Rs. %.2f\n", discount);
```

```
17      printf("GST       : Rs. %.2f\n", gst);
18      printf("Total     : Rs. %.2f\n", total);
19      printf("Net payable : Rs. %.2f\n", net);
20
21      return 0;
22  }
```

Run Code

OUTPUT:



Q2. Short-Circuit Evaluation at a Ticket Counter

Eligibility: A passenger qualifies if (senior citizen AND eligible ticket class) OR (has disability card AND the card is server-verified).

Short-circuit rules

In A && B, B is skipped when A is false. In A || B, B is skipped when A is true. C evaluates left to right. Put inexpensive checks before a slow function when skipping that function is allowed.

Recommended condition and call-count analysis

Recommended condition: (senior && eligible_class) || (has_card && verify_card()). If the senior/class condition is true, card verification is skipped. Otherwise the card flag is checked, and server verification happens only if a card is present.

For 1,000 passengers, calls in the recommended order equal the number for whom (senior AND eligible_class) is false and has_card is true. With the reversed order (has_card && verify_card()) || (senior && eligible_class), calls equal the number with a card, because verification occurs before the senior route is tested. An exact numeric count requires the passenger data.

Changed condition: audit every passenger

If every passenger's check must be audited, call the verification/audit routine explicitly for every passenger before evaluating the concession condition. This guarantees 1,000 calls for 1,000 passengers.

C program

```c
#include <stdio.h>
int slow_calls = 0;

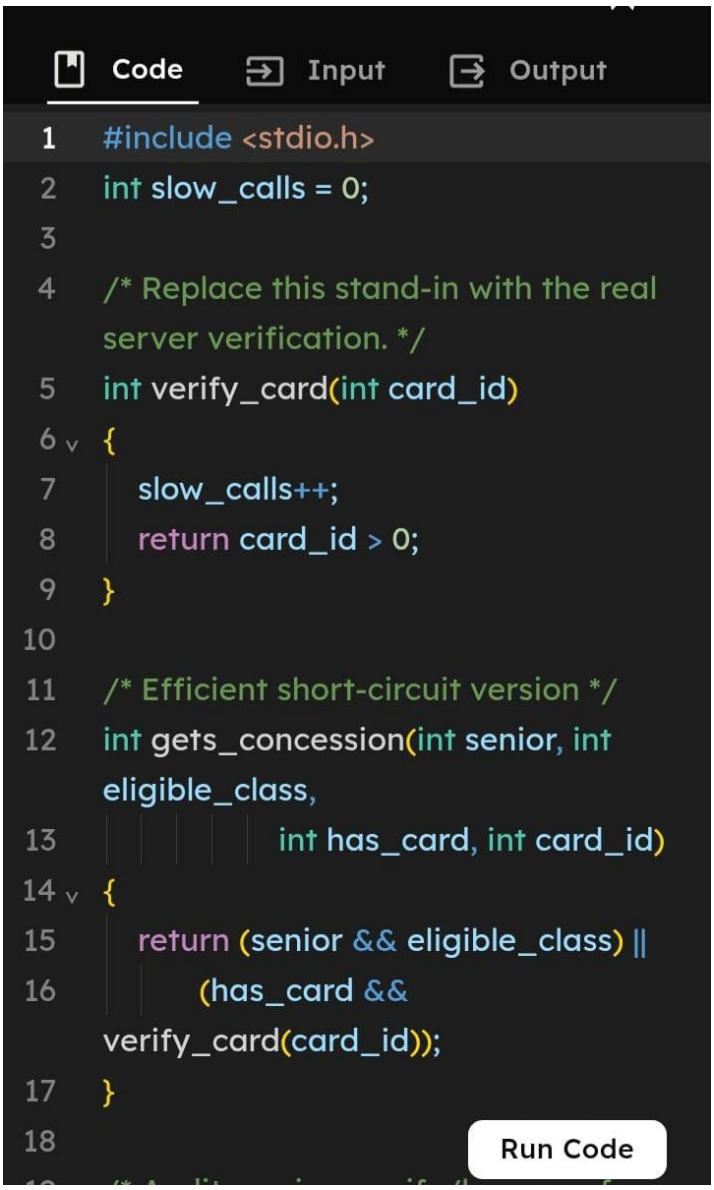
/* Replace this stand-in with the real server verification. */
int verify_card(int card_id)
{
    slow_calls++;
    return card_id > 0;
}

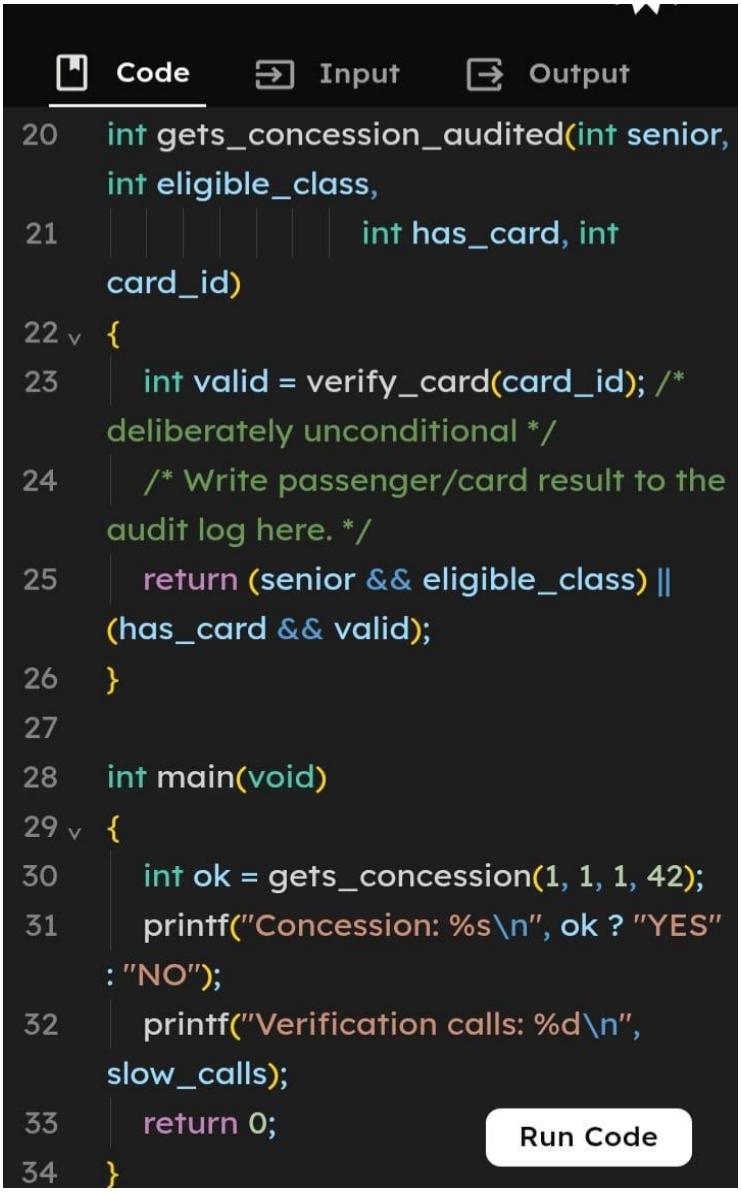
/* Efficient short-circuit version */
int gets_concession(int senior, int eligible_class,
                    int has_card, int card_id)
{
    return (senior && eligible_class) ||
           (has_card && verify_card(card_id));
}

/* Audit version: verify/log once for every passenger */
int gets_concession_audited(int senior, int eligible_class,
                            int has_card, int card_id)
{
    int valid = verify_card(card_id); /* deliberately unconditional */
    /* Write passenger/card result to the audit log here. */
    return (senior && eligible_class) || (has_card && valid);
}

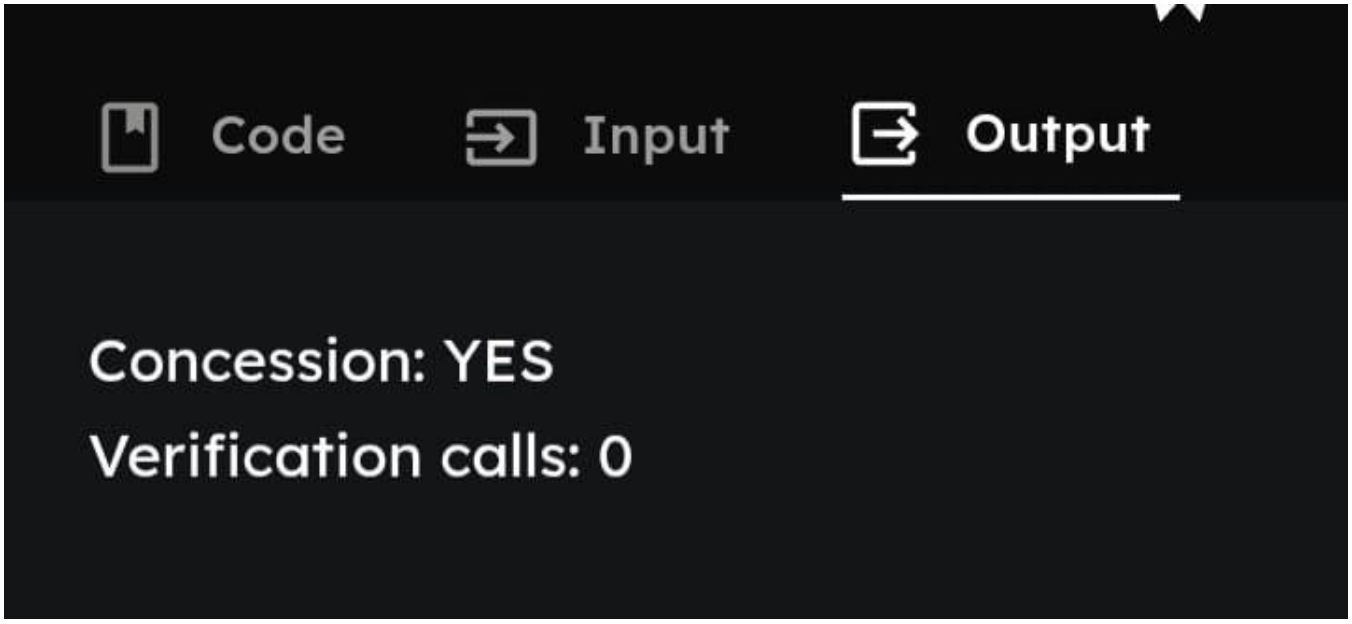
int main(void)
{
    int ok = gets_concession(1, 1, 1, 42);
    printf("Concession: %s\n", ok ? "YES" : "NO");
    printf("Verification calls: %d\n", slow_calls);
    return 0;
}
```

EXECUTED CODE:





OUTPUT:



Note: The first function is the efficient version; the audited function is for the changed requirement. A real audit implementation should record passenger identity, timestamp, card ID, and verification result.

3. Device-Status Register Using Bitwise Operations

Device bits: LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4. A set bit (1) means ON; a clear bit (0) means OFF. Five device states fit in one unsigned byte.

Bitwise operations and register trace

Set: reg | (1u << dev). Clear: reg & ~(1u << dev). Toggle: reg ^ (1u << dev). Test: (reg & (1u << dev)) != 0.

Start 00000000. Set LIGHT → 00000001. Set AC → 00000101. Toggle FAN → 00000111. Clear LIGHT → 00000110. Therefore AC is ON and PUMP is OFF. These operations update only the selected bit. Use valid bit indices 0–7.

C program

```
#include <stdio.h>
#define LIGHT 0
#define FAN 1
#define AC 2
#define HEATER 3
#define PUMP 4

unsigned char switch_on(unsigned char reg, int dev)
{ return (unsigned char)(reg | (1u << dev)); }

unsigned char switch_off(unsigned char reg, int dev)
{ return (unsigned char)(reg & ~(1u << dev)); }

unsigned char toggle(unsigned char reg, int dev)
{ return (unsigned char)(reg ^ (1u << dev)); }

int is_on(unsigned char reg, int dev)
{ return (reg & (1u << dev)) != 0; }

void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--)
        putchar((reg & (1u << bit)) ? '1' : '0');
    putchar('\n');
}

int main(void)
{
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    show(reg);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n",
           is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    return 0;
}
```

EXECUTED CODE:



Code Input Output

```c
#include <stdio.h>
#define LIGHT 0
#define FAN 1
#define AC 2
#define HEATER 3
#define PUMP 4

unsigned char switch_on(unsigned char reg, int dev)
{
    return (unsigned char)(reg | (1u << dev));
}

unsigned char switch_off(unsigned char reg, int dev)
{
    return (unsigned char)(reg & ~(1u << dev));
}

unsigned char toggle(unsigned char
```

Run Code

Code Input Output

```c
unsigned char toggle(unsigned char reg, int dev)
{
    return (unsigned char)(reg ^ (1u << dev));
}

int is_on(unsigned char reg, int dev)
{
    return (reg & (1u << dev)) != 0;
}

void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--)
        putchar((reg & (1u << bit)) ? '1' : '0');
    putchar('\n');
}

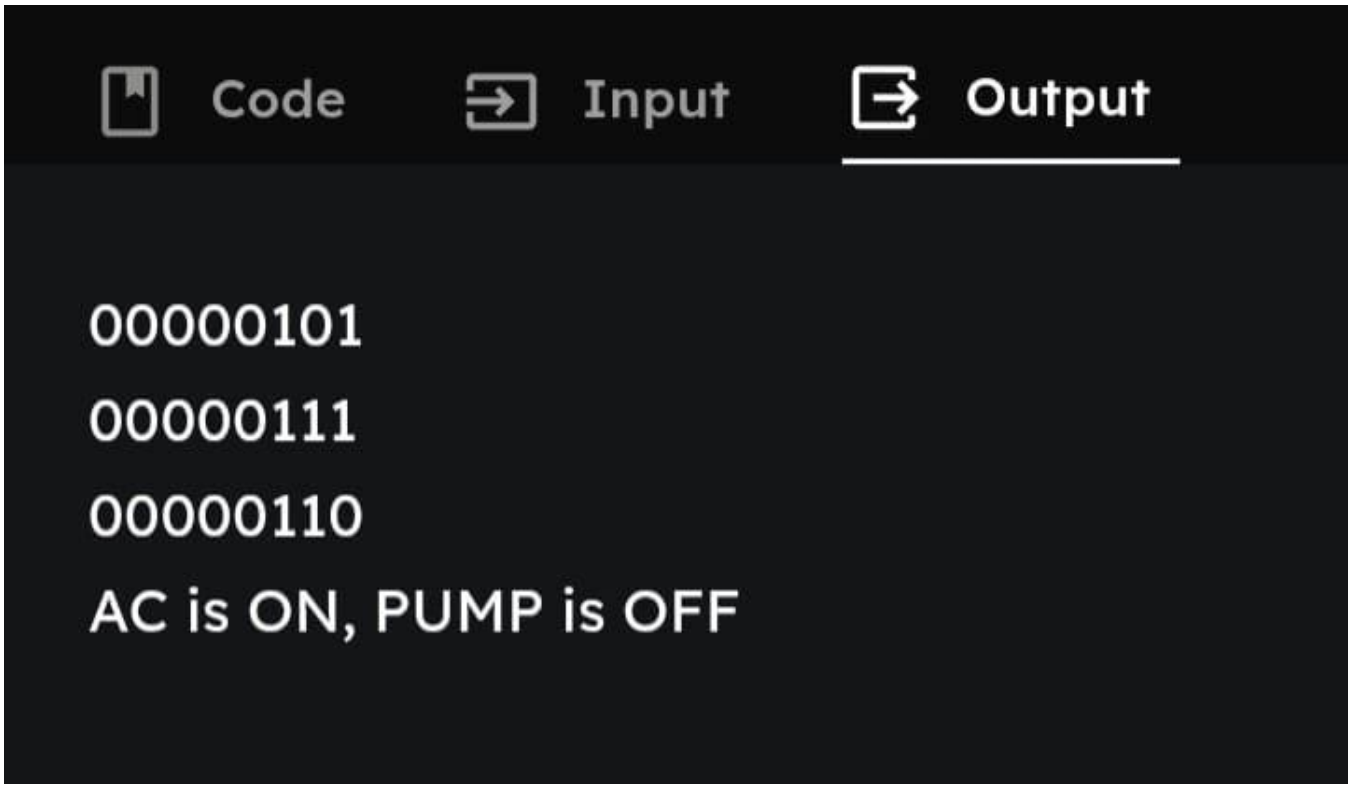
int main(void)
{
    unsigned char reg = 0;
```

Run Code

```
37    unsigned char reg = 0;
38    reg = switch_on(reg, LIGHT);
39    reg = switch_on(reg, AC);
40    show(reg);
41
42    reg = toggle(reg, FAN);
43    show(reg);
44
45    reg = switch_off(reg, LIGHT);
46    show(reg);
47
48    printf("AC is %s, PUMP is %s\n",
49        is_on(reg, AC) ? "ON" : "OFF",
50        is_on(reg, PUMP) ? "ON" : "OFF");
51
52    return 0;
53 }
```

Run Code

OUTPUT:



zQ3. Alternative Representation and Complexity

Array alternative

An array such as int state[5] stores one integer per device. It is easier to extend and understand, but it consumes more memory than a packed bit register.

Memory comparison

Assuming a typical 4-byte int, five array elements use 20 bytes per controller. The packed register uses 1 byte for up to 8 devices. For 1,000,000 controllers, this is approximately 20,000,000 bytes (20 MB decimal) for the int array versus 1,000,000 bytes (1 MB decimal) for the register, excluding other overhead. If the array uses one byte per device, it uses 5 MB for one million controllers.

Time complexity and conclusion

Setting, clearing, toggling, and testing one known bit each take O(1). Counting ON devices checks five bits, O(5), which is O(1) for this fixed-size set. An array also provides O(1) access by index, but counting ON states requires scanning all five elements. A bit register is compact and efficient for Boolean states; an array is simpler when each device needs more information or the number of devices changes frequently.